

Reviewer Pack — Minimal Rank of Quadratic Forms Bounds Monomial Circuit Size...

Entry eeb89688e704 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 11:04:08 UTC

Minimal Rank of Quadratic Forms Bounds Monomial Circuit Size for XOR 3-CNF

Entry ID: eeb89688e704 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 11:04:08 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Forms) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For all instances of XOR 3-CNFs with n variables, the minimal rank of their associated quadratic form is at least proportional to the size of the smallest monomial circuit that can decide the satisfiability of the instance.’, ‘Specifically, there exists a constant $c > 0$ such that for any XOR 3-CNF instance F with n variables, the minimal rank of its quadratic form $q(F)$ satisfies $q(F) \geq cn$.’, ‘This proportionality holds even when considering only circuits composed of AND and XOR gates.’]

Rationale (proposer’s reasoning):

[‘Quadratic forms have been studied in algebraic geometry for their geometric properties. These properties are often captured by the minimal rank, which can reflect the complexity of the underlying problem.’, ‘In circuit complexity, monomial circuits are a natural generalization of AND-OR circuits that include XOR gates. Their size is related to the difficulty of computing a function with those gates.’, ‘Linking these two fields through a quantitative relation could potentially reveal new insights into the computational hardness of satisfiability problems.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ad5d4f334104141f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the ratio of the minimal rank of the quadratic form to the size of the monomial circuit for all generated XOR 3-CNF instances is greater than or equal to a predefined constant c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=17.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_xor_3cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = []
            for i in range(n):
                if random.choice([True, False]):
                    clause.append(f"x{i+1}")
                else:
                    clause.append(f"~x{i+1}")
            clauses.append(clause)
        return clauses

    def quadratic_form(clauses):
        n = len(clauses[0])
        q = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in clauses:
            for literal in clause:
                if literal.startswith('x'):
                    var = int(literal[1:]) - 1
                    q[var][var] += 1
                elif literal.startswith('~x'):
                    var = int(literal[2:]) - 1
                    q[var][var] -= 1
        return q

    def gaussian_elimination(A):
```

```

n = len(A)
for i in range(n):
    max_row = i
    for j in range(i + 1, n):
        if abs(A[j][i]) > abs(A[max_row][i]):
            max_row = j
    A[i], A[max_row] = A[max_row], A[i]
    if A[i][i] == 0:
        return None
    for j in range(i + 1, n):
        factor = -A[j][i] / A[i][i]
        for k in range(n + 1):
            A[j][k] += factor * A[i][k]
rank = sum(1 for row in A if any(row))
return rank

def monomial_circuit_size(clauses):
    n = len(clauses[0])
    size = 0
    for clause in clauses:
        size += len(clause) - 1
    return size

n = random.choice([5, 10, 15, 20, 30, 40])
instance = generate_xor_3cnf(n)
q_form = quadratic_form(instance)
rank = gaussian_elimination(q_form)
if rank is None:
    return {
        "metric_name": "rank_to_circuit_size_ratio",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "singular_matrix"
    }
circuit_size = monomial_circuit_size(instance)
ratio = Fraction(rank, circuit_size)

return {
    "metric_name": "rank_to_circuit_size_ratio",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

metric_values = [r["metric_value"] for r in results]
conjecture_holds = all(r["conjecture_holds"] for r in results)

if conjecture_holds:
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"rank_to_circuit_size_ratio\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cfc71e7a.py", line 99, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cfc71e7a.py", line 73, in run_trial
    rank = gaussian_elimination(q_form)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cfc71e7a.py", line 59, in gaussian_elimination
    A[j][k] += factor * A[i][k]
    ~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the conjecture could neither be supported nor falsified based on the available information. | next: Re-run the test to ensure it completes successfully and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16669
2	preregistration	ollama_remote	glm4:latest	0	0	5697
3	novelty	ollama_remote	glm4:latest	0	0	4886
4	novelty	ollama_remote	glm4:latest	0	0	5224
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17470
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11192
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14424
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10816
9	judge	ollama_remote	glm4:latest	0	0	9254

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 95633 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/eeb89688e704.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eeb89688e704.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/eeb89688e704.tar.gz` (if generated)